

Jacobi 集实现：串行版与 mycode 核心不同点对照

本文档对照 CODE/JacobiSetComputation-master（串行）与 CODE/mycode（CUDA 并行）在 Jacobi 集判定上的**相同逻辑与核心差异**，并给出具体代码位置，便于跳转阅读。

一、相同部分：判定公式本身（三处一致）

这三处逻辑相同，只是实现位置不同。

1. 边级最终判定： `f_crit` $\rightarrow$ 属于 Jacobi 集

位置	文件	行号
串行	JacobiSet.cpp	240–253
GPU kernel	jacobi_cuda.cu	135–142, 150
SoS 回退	main_cuda.cpp	117–129

串行（CODE/JacobiSetComputation-master/src/JacobiSet.cpp）：

```
bool f_lower_v1 = is_lowerLink(e1, e2, v1, g);
bool f_lower_v2 = is_lowerLink(e1, e2, v2, g);
bool g_lower_v1 = is_lowerLink(e1, e2, v1, f);
bool g_lower_v2 = is_lowerLink(e1, e2, v2, f);

bool f_crit = (f_lower_v1 == f_lower_v2);
bool g_crit = (g_lower_v1 == g_lower_v2);

if(f_crit){
    jacobiedges.push_back(JacobiEdge(e1, e2, !f_lower_v1, g_lower_v1, alignment(e1, e2)));
}
```

mycode GPU kernel（CODE/mycode/jacobi_cuda.cu）：

```
const bool f_lower_v1 = device_is_lower_link(edge.e1, edge.e2, edge.link1, f, g, g, &degenerate);
const bool f_lower_v2 = device_is_lower_link(edge.e1, edge.e2, edge.link2, f, g, g, &degenerate);
const bool g_lower_v1 = device_is_lower_link(edge.e1, edge.e2, edge.link1, f, g, f, &degenerate);
const bool g_lower_v2 = device_is_lower_link(edge.e1, edge.e2, edge.link2, f, g, f, &degenerate);

const bool f_crit = (f_lower_v1 == f_lower_v2);
const bool g_crit = (g_lower_v1 == g_lower_v2);
const bool is_jacobi = f_crit;
```

判定含义：link 中 v1、v2 关于 f（以 g 为限制）的 lower-link 分类相同，即 f_lower_v1 == f_lower_v2，则该边属于 Jacobi 集。

2. lower-link 判定： POS1 == POS2

位置	文件	行号
串行	JacobiSet.cpp	191–205
GPU	jacobi_cuda.cu	85–108

串行：

```
bool is_POS1 = POS1(a,b,v);
if(index_j == (index_i+1)%3) is_POS2 = POS2(i, j, *F);
else is_POS2 = POS2(j, i, *F);

return (is_POS1 == is_POS2);
```

mycode：

```
const bool is_pos1 = device_pos1(a, b, v, f, g, degenerate);
// ... POS2 分支 ...
return is_pos1 == is_pos2;
```

3. POS1 / POS2 / alignment 子函数

函数	串行	GPU
POS1	JacobiSet.cpp 147–165	jacobi_cuda.cu 43–71
POS2	JacobiSet.cpp 170–180	jacobi_cuda.cu 74–82
alignment	JacobiSet.cpp 136–142	jacobi_cuda.cu 111–119

二、核心不同点 1：执行方式（串行循环 vs GPU 并行）

串行：一条边接一条边算

位置： JacobiSet.cpp 第 226 行

```
for(std::set<trimesh::Edge>::const_iterator iter = mesh->edges.begin();
    iter != mesh->edges.end(); iter++){
```

mycode：一个线程处理一条边

Kernel 线程映射： jacobi_cuda.cu 第 127–130 行

```
const int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx >= edge_count) {
    return;
}
```

Kernel 启动： jacobi_cuda.cu 第 226–234 行

```
const int block_size = 256;
const int grid_size = static_cast<int>((edges.size() + block_size - 1) / block_size);
compute_jacobi_kernel<<<grid_size, block_size>>>(d_edges, ...);
```

主程序调用 GPU： main_cuda.cpp 第 235–241 行

```

if (!compute_jacobi_gpu(interior_edges, f, g, &results, &timing, &error)) {
    std::cerr << "GPU computation failed: " << error << "\n";
    return EXIT_FAILURE;
}

```

三、核心不同点 2：拓扑输入（循环里查 link vs 预展平 EdgeRecord）

串行：遍历时临时调用 `get_e_link()`

位置：JacobiSet.cpp 第 228–233 行

```

int e1 = iter->first;
int e2 = iter->second;

std::pair<int,int> lnk = mesh->get_e_link(*iter);
int v1 = lnk.first;
int v2 = lnk.second;

```

`get_e_link()` 实现在 TriMeshJ.cpp（串行工程）。

mycode：CPU 预处理生成 `EdgeRecord[]`，GPU 只读数组

结构定义：jacobi_gpu.h 第 7–12 行

```

struct EdgeRecord {
    int e1;
    int e2;
    int link1;
    int link2;
};

```

预处理：main_cuda.cpp 第 197–208 行

```

for (std::set<trimesh::Edge>::const_iterator it = mesh.edges.begin();
     it != mesh.edges.end(); ++it) {
    std::pair<int, int> link = mesh.get_e_link(*it);
    if (link.first == -1 || link.second == -1) {
        continue;
    }
    EdgeRecord record;
    record.e1 = it->first;
    record.e2 = it->second;
    record.link1 = link.first;
    record.link2 = link.second;
    interior_edges.push_back(record);
}

```

GPU 读取： jacobi_cuda.cu 第 132 行

```

const EdgeRecord edge = edges[idx];

```

四、核心不同点 3：退化处理（全程 SoS vs GPU 标记 + CPU 回退）

串行：判定全程走 SoS（启用 USE_SOS 时）

POS1 SoS 分支： JacobiSet.cpp 第 149–150 行

```

#ifdef USE_SOS
    return sos_lambda3(a+1,b+1,v+1)->signum < 0;

```

mycode 的 CMake 也定义了 USE_SOS，但仅用于 CPU 回退路径（CMakeLists.txt 第 34 行）。

mycode GPU：double 近似 + 退化标记

阈值： jacobi_cuda.cu 第 9 行

```

constexpr double kEps = 1e-12;

```

POS1 退化标记 ($X \approx 0$): jacobi_cuda.cu 第 54–58 行

```
if (!device_are_equal(x, 0.0)) {  
    return x > 0.0;  
}  
*degenerate = 1;
```

POS2 退化标记 (函数值近似相等): jacobi_cuda.cu 第 78–80 行

```
if (device_are_equal(field[a], field[b])) {  
    *degenerate = 1;  
    return a < b;  
}
```

Kernel 汇总退化: jacobi_cuda.cu 第 151 行

```
out.degenerate = degenerate || (f_crit != g_crit);
```

mycode CPU 回退：只对退化边调用串行 SoS

回退函数: main_cuda.cpp 第 99–137 行

```
static size_t apply_sos_fallback(...) {  
    JacobiSet js(&mesh, &f, &g);  
    for (size_t i = 0; i < edges.size(); ++i) {  
        if (!(*results)[i].degenerate) {  
            continue;  
        }  
        const bool f_lower_v1 = js.is_lowerLink(edge.e1, edge.e2, edge.link1, &g);  
        // ... 覆盖 out.is_jacobi 等字段 ...  
    }  
}
```

主流程调用: main_cuda.cpp 第 243 行

```
const size_t sos_fallback_count = apply_sos_fallback(mesh, f, g, interior_edges, &results);
```

策略: 普通边走 GPU 快路径；退化边走 CPU SoS 精确路径，保证与串行结果一致。

五、核心不同点 4：结果输出（动态 push_back vs 固定槽位 + 过滤）

串行：满足条件才 push_back

位置：JacobiSet.cpp 第 252–253 行

```
if(f_crit){  
    jacobiedges.push_back(JacobiEdge(e1, e2, !f_lower_v1, g_lower_v1, alignment(e1, e2)));  
}
```

数据结构：JacobiSet.h 第 21–33 行（JacobiEdge）

mycode：每条边固定槽位，写文件时再过滤

结果结构：jacobi_gpu.h 第 14–22 行（JacobiGpuResult）

Kernel 写入固定位置：jacobi_cuda.cu 第 152 行

```
results[idx] = out;
```

写文件时按 is_jacobi 过滤：main_cuda.cpp 第 52–70 行

```
for (size_t i = 0; i < results.size(); ++i) {  
    if (results[i].is_jacobi) {  
        ++count;  
    }  
}  
// ...  
for (size_t i = 0; i < results.size(); ++i) {  
    if (!results[i].is_jacobi) {  
        continue;  
    }  
    // 写出 Jacobi 边  
}
```

六、核心不同点 5：整体流程分工（CPU-GPU 异构）

串行全流程（CPU）

位置： main.cpp 第 117–122 行

```
JacobiSet js(mesh, &f, &g);
js.compute();
js.write(outfile);
```

mycode 分阶段流程

阶段	代码位置
读 mesh + 建边表	main_cuda.cpp 190–209
构造 f=y, g=x	main_cuda.cpp 228–233
GPU 并行判定	main_cuda.cpp 238 → jacobi_cuda.cu 157–260
CPU SoS 回退	main_cuda.cpp 243
写结果	main_cuda.cpp 256–257

```
读 mesh
→ CPU 预处理 EdgeRecord[]
→ cudaMemcpy 到 GPU
→ CUDA kernel（一线程一边）
→ cudaMemcpy 回 CPU
→ CPU SoS 回退（仅 degenerate 边）
→ 写 *_jacobi.txt
```

七、快速对照表

差异点	串行代码位置	mycode 代码位置
判定公式	JacobiSet.cpp:240-253	jacobi_cuda.cu:135-142

差异点	串行代码位置	mycode 代码位置
执行方式	JacobiSet.cpp:226 （for 循环）	jacobi_cuda.cu:127-130 （thread idx）
link 获取	JacobiSet.cpp:232 （ get_e_link ）	main_cuda.cpp:197-208 （预处理）
退化处理	JacobiSet.cpp:149-150 （SoS）	GPU: jacobi_cuda.cu:58,79,151 ； 回退: main_cuda.cpp:99-129
结果收集	JacobiSet.cpp:252 （push_back）	jacobi_cuda.cu:152 + main_cuda.cpp:61-70 （过滤写）
主流程	main.cpp:117-122	main_cuda.cpp:190-257

八、相关文件索引

文件	路径
串行 Jacobi 判定	CODE/JacobiSetComputation-master/src/JacobiSet.cpp
串行主程序	CODE/JacobiSetComputation-master/src/main.cpp
CUDA 主程序	CODE/mycode/main_cuda.cpp
CUDA kernel	CODE/mycode/jacobi_cuda.cu
共享数据结构	CODE/mycode/jacobi_gpu.h
CMake （CUDA）	CODE/mycode/CMakeLists.txt